Spam Classification Logistic Regression

Maxine Somers
Emilio Wagner Munguia
Connor Tate

February 27, 2026

1 Abstract

We investigated logistic regression for spam email classification using the UCI Spambase dataset. We implemented mini-batch stochastic gradient descent (SGD) with L2 regularization and performed 5-fold cross-validation for hyperparameter tuning, selecting $\alpha = 0.1$, $B = 16$, and 50 epochs. A regularization sweep identified $\lambda^* = 0.01$ as optimal, yielding a test accuracy of 89.87% and cross-entropy of 0.3143. We additionally compared our implementation against L1-regularized logistic regression from `scikit-learn`, finding that L1 achieved comparable accuracy with greater sparsity, highlighting its advantage for feature selection and interpretability.

2 Introduction

Spam refers to unwanted digital communication. In large quantities, unwanted spam emails can become overwhelming for recipients. It is therefore of interest to have tools to automatically classify emails as spam, based on their content. In this project, we investigated logistic regression for classification of spam emails. We implemented mini-batch SGD with L2 regularization, and performed K-fold cross validation for hyperparameter tuning and model selection. We also compared our implementation to L1-regularized logistic regression from `scikit-learn`. We used the *Spambase* dataset prepared by UC Irvine as part of their Machine Learning Repository [1]. It consists of 4601 email instances. Our target `Class` is a binary identifier of spam `1 = spam` and `0 = not spam`.

3 Methods

3.1 Data Preprocessing and Exploration

We first inspected our dataset and found it consists of 4601 email instances. We explored the data types of our features, and plotted their distributions (Appendix A). We then searched the dataset for any missing values, which could cause failures or errors in our model. We used `pandas.DataFrame.dropna()` which removes NaN, None, or NaT values, but found 0 missing values. We divided our dataset into training (5%) and test (95%) sets. Finally, we applied standardization normalization, calculated from the training set.

3.2 Logistic Regression with SGD

We implemented logistic regression for binary classification using cross-entropy loss for our cost function:

$$l_{CE}(\hat{y}, y) = -y\log(\hat{y}) - (1-y)\log(1-\hat{y}) \qquad \text{where } \hat{y} = \sigma(w^\top x)$$

$$J_w = \frac{1}{N}\sum_n -y\log(\hat{y}^{(n)}) - (1-y^{(n)})\log(1-\hat{y}^{(n)})$$

We then implemented SGD in order to minimize the cost of our model. At each epoch, we shuffle the data to avoid bias. Then, we compute the gradient based on batches of size B. We added L2 regularization, excluding the bias:

$$\nabla J(w) = \sum_{x,y\in\mathcal{D}} x(w^\top x - y) + \lambda w \qquad \lambda w : \text{weight decay reduces the weights at each step}$$

We initialized weights to tiny random values, then on each iteration of SGD, we update the weights. The weights are updates by multiplying the learning rate α in the direction of the gradient.

$$w^{\{t+1\}} \leftarrow w^{\{t\}} - \alpha \nabla J(w^{\{t\}})$$

We then ran our model 30 combinations of initial hyperparameters (Table 1).

Table 1: Initial hyperparameter configurations

Hyperparameter	**Values**
Batch size B	`[1,16,64]`
Learning rate α	`[1, 1e-1, 1e-2, 1e-3, 1e-4]`
Regularization parameter λ	`1e-3`
Epochs	`100`

3.3 Hyperparameter Tuning

We applied K-fold cross-validation on our training data for hyperparameter selection. We chose to split our train data into 5 folds for compute time. For each fold we train on 4 folds and evaluate on the held-out fold. To avoid data leakage, we compute mean and std of features using only the training folds. Then, we standardize both training and validation data with those values. We trained using our logistic regression model on the training folds, and evaluate it on the validation fold using cross-entropy and accuracy measures. After K folds, we compute:

$$\text{mean cross-entropy: } \frac{1}{K}\sum_{k=1}^{K} \text{CE}_k \quad \text{where} \quad \text{CE} = \frac{1}{N}\sum_n -y^{(n)}\log(\hat{y}^{(n)}) - (1-y^{(n)})\log(1-\hat{y}^{(n)})$$

$$\text{mean accuracy: } \frac{1}{K}\sum_{k=1}^{K} \text{acc}_k \quad \text{where} \quad \text{acc} = \frac{1}{N}\sum_n \mathbb{I}[\hat{y}^{(n)} = y^{(n)}]$$

$$\text{SEM cross-entropy: } \frac{\sigma_{\text{CE}}}{\sqrt{K}} \quad \text{where} \quad \sigma_{\text{CE}} = \sqrt{\frac{1}{K}\sum_{k=1}^{K}(\text{CE}_k - \overline{\text{CE}})^2}$$

$$\text{SEM accuracy: } \frac{\sigma_{\text{acc}}}{\sqrt{K}} \quad \text{where} \quad \sigma_{\text{acc}} = \sqrt{\frac{1}{K}\sum_{k=1}^{K}(\text{acc}_k - \overline{\text{acc}})^2}$$

Our grid search runs over 100 combinations of hyperparameters (Table 2). Results are saved to `cv_hyperparameter_results.csv`.

Table 2: Hyperparameter configurations for tuning

Hyperparameter	**Values**
Batch size B	`[1,8,16,32,64]`
Learning rate α	`[1, 1e-1, 1e-2, 1e-3, 1e-4]`
Regularization parameter λ	`1e-3`
Epochs	`[50,100,150,200]`

After running, we select our hyperparameter configuration. First we consider all configurations within one standard error of the best mean accuracy score. Then, between those, we choose the simplest model, preferring fewer epochs, and larger batch sizes.

3.4 Bias–Variance Trade-Off

We demonstrated bias-variance behavior by sweeping L2 regularization strength and evaluating with K-fold CV (K=5). Other than regularization parameter λ, we used fixed hyperparameters to allow the model to fit training data well (Table 3). These hyperparameters allow fast convergence, so that overfitting becomes clearly visible.

Table 3: Hyperparameter configurations for lambda sweep

Hyperparameter	**Values**
Batch size B	`16`
Learning rate α	`0.1`
Regularization parameter λ	`[0, 1e-6, 1e-5, 1e-4, 1e-3, 1e-2, 1e-1, 1]`
Epochs	`200`

For each λ, we ran K-fold, similar to Section 3.3. The mean and standard error of the cross-entropy (CE) and accuracy (acc) are recorded across folds (refer to Section 3.3 for equations). The optimal λ^* is selected by minimising validation CE, with ties broken by maximising validation accuracy and then preferring the larger λ: Finally, we retrained on the full training set using λ^*, with standardization fit on the full training set and applied to the held-out test set. Test CE and accuracy are reported as the final performance metrics.

3.5 L1-Regularized Logistic Regression and the Regularization Path

To study sparsity and feature selection, we trained L1-regularized logistic regression models using scikit-learn's implementation with the `saga` solver, which supports L1 penalties. Features were standardized prior to training to ensure consistent regularization across coefficients. We evaluated model behavior across a range of regularization strengths (λ values). For each λ: (1) The model was trained on the training set. (2) Training and validation cross-entropy loss were computed. (3) Classification accuracy was measured. (4) The number of non-zero coefficients was recorded to quantify sparsity. To assess generalization performance, we used cross-validation. We then plotted: (1) Validation performance versus regularization strength. (2)Number of non-zero coefficients versus regularization strength. (3) Coefficient paths as λ varied.

4 Results

4.1 Logistic Regression with SGD

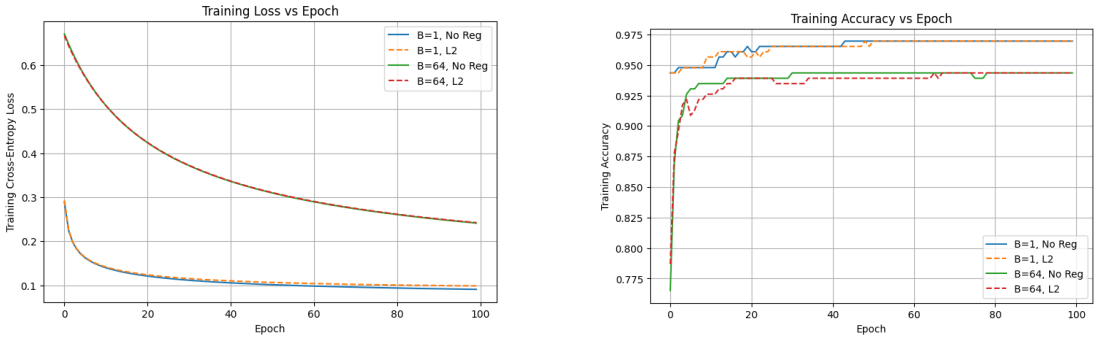

Figure 1: Training curves for regularized and unregularized Logistic Regression model, showing training cross-entropy and accuracy.

4.2 Hyperparameter Tuning

All 100 hyperparameter configurations were plotted (Figure 2), and the top 20 were selected for display (Appendix A: Table 5).

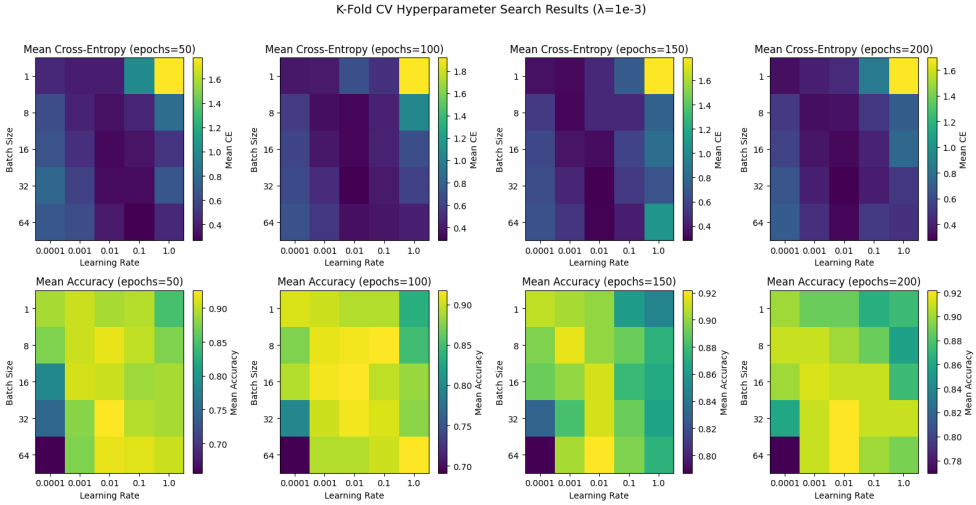

Figure 2: Mean CE and mean accuracy for CV hyperparameter sweep

The chosen hyperparameter configuration was $\alpha = 0.1$, $B = 16$, and Epochs= 50. This configuration had a mean CE of 0.3620 and a mean accuracy $\pm$ SEM of 0.9217 ± 0.0132

4.3 Bias–Variance Trade-Off

From K-fold λ sweep, we plotted CE and accuracy (Figure 3). Our selected λ from CV was $\lambda = 0.01$ with a $CE = 0.2958$ and $acc = 0.9000$.

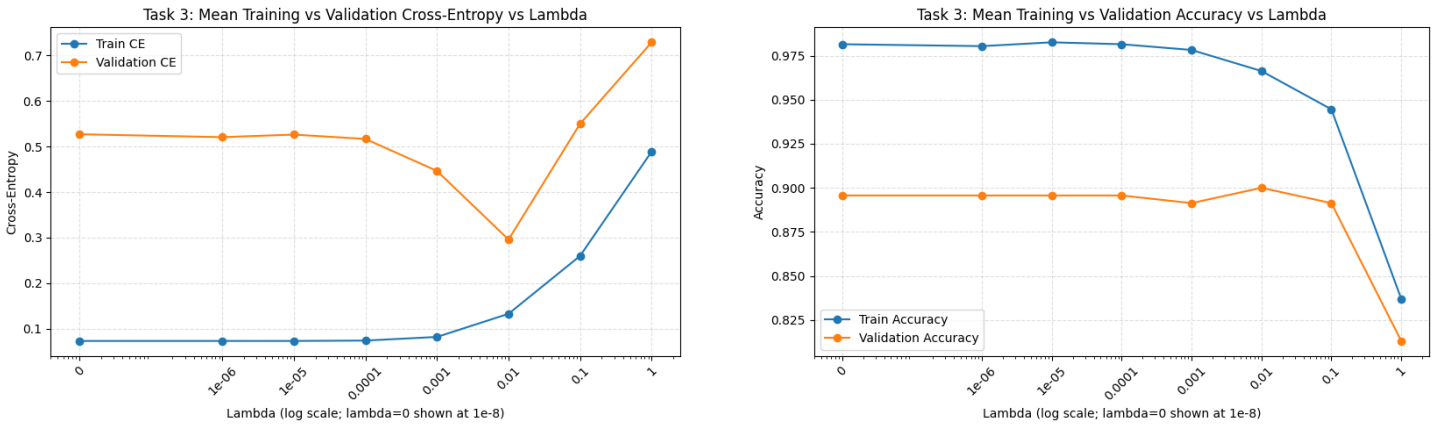

Figure 3: Lambda sweep plots for CE and accuracy

Table 4: Cross-validation results across the lambda sweep

Lambda	Train CE Mean	Val CE Mean	Train Acc Mean	Val Acc Mean	Train CE Std	Val CE Std	Train Acc Std	Val Acc Std
0	0.0732	0.5271	0.9815	0.8957	0.0134	0.2888	0.0055	0.0320
10^{-6}	0.0731	0.5207	0.9804	0.8957	0.0134	0.2871	0.0055	0.0320
10^{-5}	0.0732	0.5263	0.9826	0.8957	0.0133	0.2881	0.0041	0.0320
10^{-4}	0.0740	0.5167	0.9815	0.8957	0.0133	0.2833	0.0055	0.0320
10^{-3}	0.0820	0.4465	0.9783	0.8913	0.0127	0.2311	0.0077	0.0275
10^{-2}	0.1328	0.2958	0.9663	0.9000	0.0100	0.0919	0.0094	0.0222
10^{-1}	0.2599	0.5501	0.9446	0.8913	0.0067	0.4885	0.0121	0.0364
1	0.4889	0.7289	0.8370	0.8130	0.0111	0.4706	0.0294	0.0593

After selecting $\lambda = 0.01$, we evaluated on our test set and found Test CE= 0.3143 and Test accuracy= 0.8987

4.4 L1-Regularized Logistic Regression and the Regularization Path

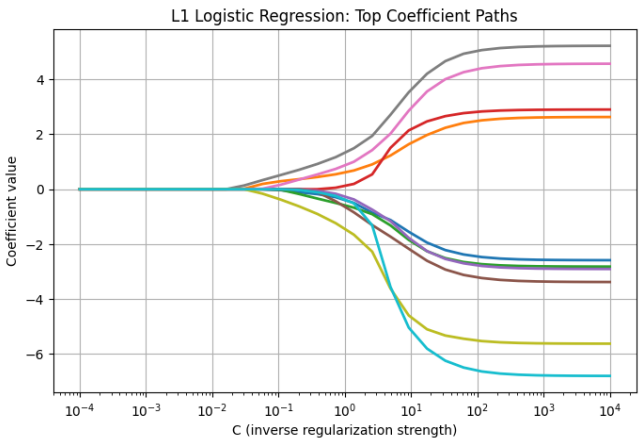

Figure 4: Coefficient Paths as λ varies

With strong regularization (small C), most coefficients are exactly zero, resulting in a highly sparse model. As C increases, more coefficients become non-zero, increasing model complexity.

Features that become non-zero earliest and grow consistently across the path appear to be the most important, as they are selected even under strong regularization.

Sparse models at small C are easier to interpret but may underfit. As regularization weakens, performance typically improves up to a point, after which additional features add little predictive value while reducing interpretability.

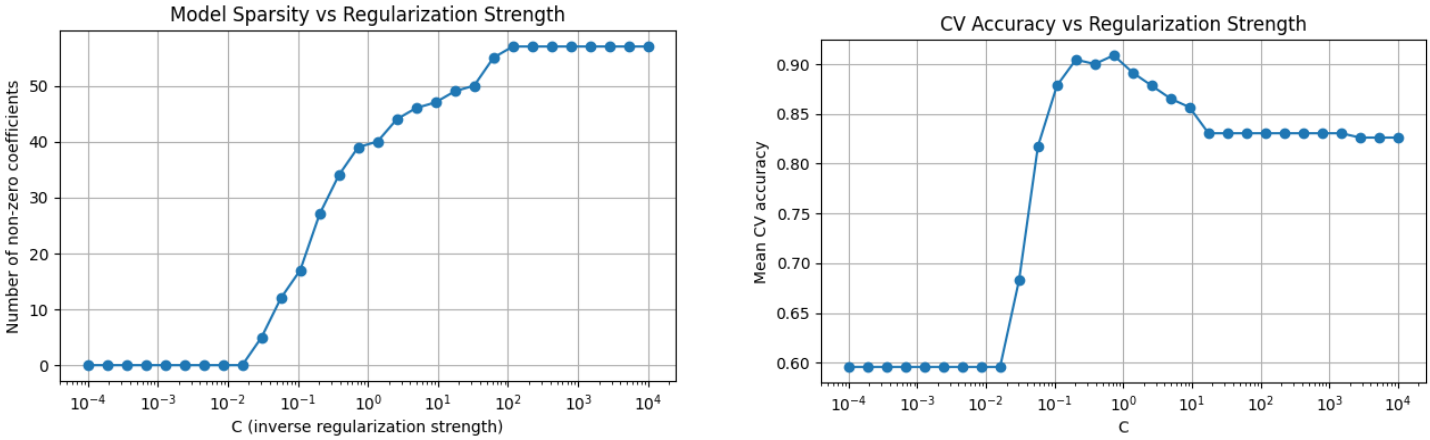

Figure 5: Regularization strength's effect on model sparsity and accuracy

5 Discussion and Conclusion

1. **Optimization behavior (SGD vs. mini-batch):** SGD ($B = 1$) produced noisier updates and less stable convergence due to high-variance gradient estimates. In contrast, mini-batch training reduced gradient variance and led to smoother learning curves and more consistent final performance. Mid-sized batch sizes offered a good balance between stability and computational efficiency.

2. **Hyperparameter selection:** Cross-validation selected $\alpha = 0.1$, $B = 16$, and 50 epochs. While slightly higher accuracies were achieved with more computationally expensive configurations, the chosen setting achieved comparable performance with lower complexity, making it a practical and efficient choice.

3. **Impact of hyperparameters:** The learning rate had the strongest effect on model performance. Very large values led to unstable or poor convergence, while moderate values yielded the best results. Batch size had a smaller influence, though mid-sized batches generally provided more reliable validation performance.

4. **Bias–variance trade-off (λ sweep):** The regularization sweep clearly illustrated the bias–variance trade-off. Small λ values resulted in overfitting, characterized by very low training error but poor validation performance (high variance). Large λ values caused underfitting (high bias) as weights were overly constrained. Validation error followed a U-shaped curve, with optimal performance achieved at an intermediate regularization strength.

5. **L1 regularization and sparsity:** The L1 regularization path showed how coefficients evolved as the regularization strength changed. Strong regularization drove many coefficients exactly to zero, producing sparse and interpretable models. As regularization weakened, more features entered the model and coefficients increased in magnitude.

6. **Comparison of L2 vs. L1 behavior:** L2 regularization shrinks all coefficients smoothly toward zero, while L1 induces exact sparsity by setting less informative coefficients to zero. In our experiments, L1 achieved comparable accuracy using fewer active features, highlighting its usefulness for interpretability and implicit feature selection, whereas L2 retained all features with small but non-zero weights.

7. **Limitations and future work:** A primary limitation was the small training subset, which may have increased variance in cross-validation estimates. Additionally, the dataset reflects a single user's email distribution, which may limit generalizability. Future work could incorporate more diverse training data and explore nonlinear models, such as gradient-boosted trees, to better capture potential feature interactions.

6 Statements

- **Resource usage**: Cursor helped for Task 2, specifically managing csv data and populating LaTeXtable
- **Team**: All team members contributed equally

7 Appendix A

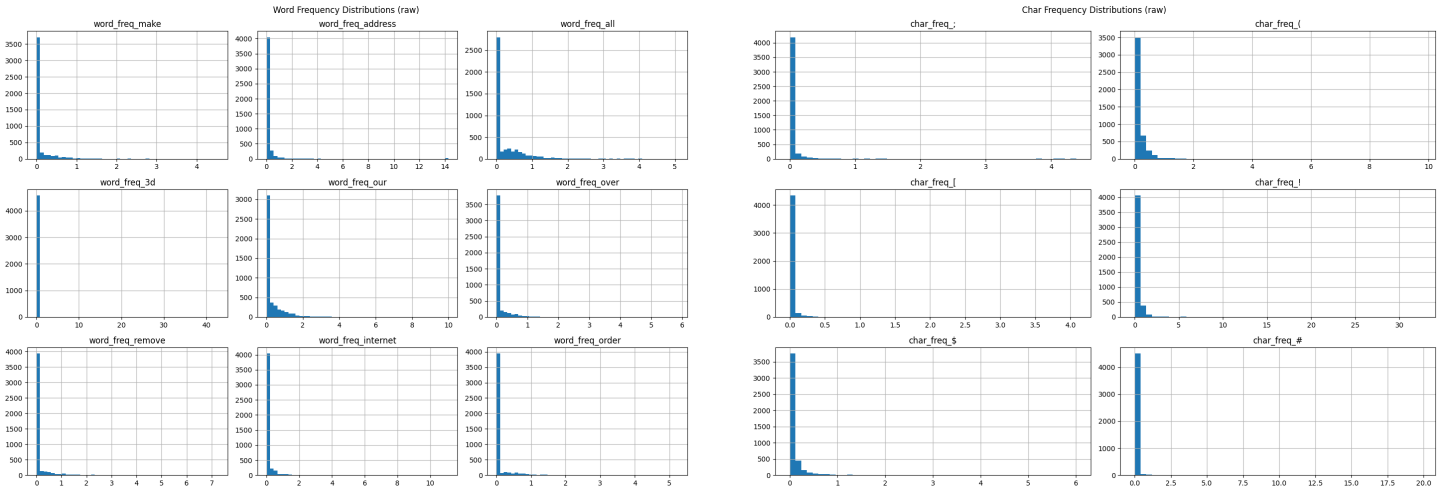

Figure 6

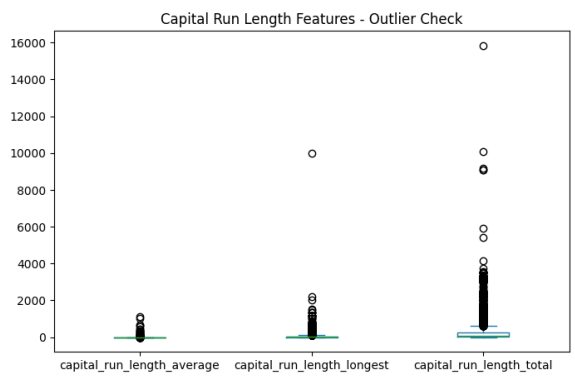

Figure 7: Caption

Our features consist of

- 48 continuous [0,100] attributes `word_freq_WORD` = percentage of words in e-mail that match WORD
- 6 continuous [0,100] attributes `char_freq_CHAR` = percentage of characters in e-mail that match CHAR
- `capital_run_length_average` which is a continuous [1,...] attribute representing the average length of uninterrupted sequences of capital letters
- `capital_run_length_longest` which is a continuous integer [1,...] attribute representing the length of longest uninterrupted sequence of capital letters
- `capital_run_length_total` which is a continuous integer [1,...] attribute representing sum of length of uninterrupted sequences of capital letters (total number of capital letters in the e-mail)
- Our target `Class`, a binary identifier of spam `1 = spam` and `0 = not spam`.

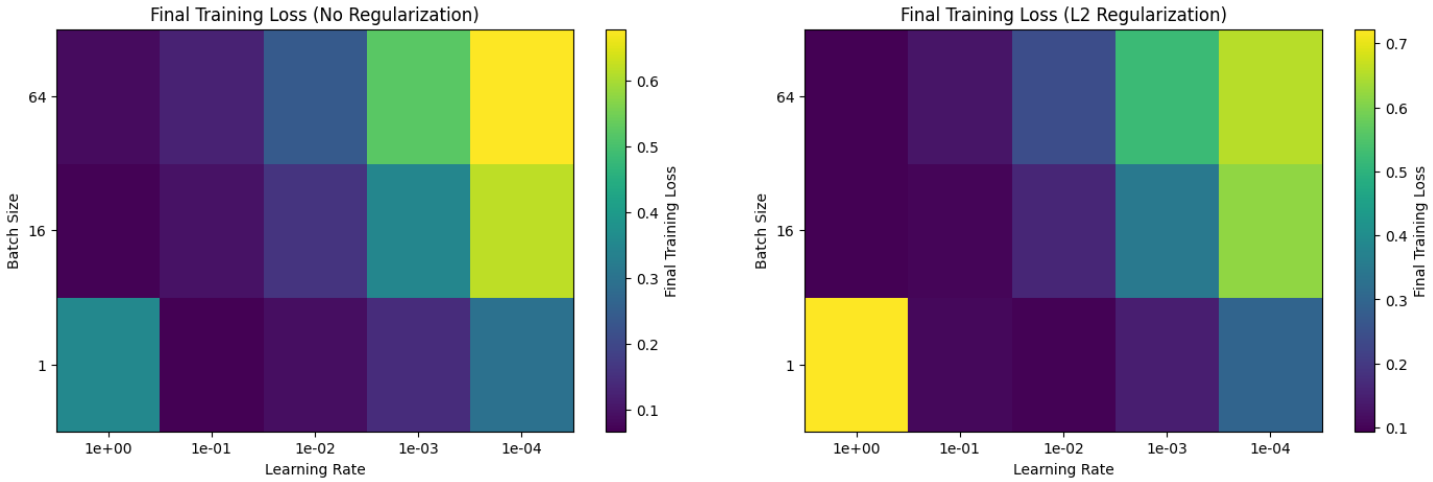

Figure 8: Training loss of initial regularized and unregularized Logistic Regression mode

Table 5: Top 20 hyperparameter configurations by mean accuracy (5-fold CV). Selected configuration (by 1-SE rule) highlighted.

Rank	Learning Rate	Batch Size	Epochs	Mean CE	Mean Acc ± SEM
1	0.01	32	50	0.3093	0.9261 ± 0.0099
2	0.01	32	200	0.2747	0.9217 ± 0.0170
3	0.01	64	150	0.2898	0.9217 ± 0.0180
4	0.01	64	200	0.3016	0.9217 ± 0.0132
5	1.0	64	100	0.4259	0.9174 ± 0.0208
6	0.1	8	100	0.4354	0.9174 ± 0.0129
7	0.01	8	50	0.3360	0.9174 ± 0.0113
8	0.01	16	100	0.3111	0.9174 ± 0.0233
9	0.01	64	50	0.3766	0.9174 ± 0.0095
10	0.001	8	150	0.3062	0.9174 ± 0.0156
11	0.1	64	50	0.2678	0.9130 ± 0.0087
12	0.01	8	100	0.3115	0.9130 ± 0.0123
13	0.01	16	150	0.3292	0.9130 ± 0.0254
14	0.01	32	100	0.2836	0.9130 ± 0.0194
15	0.01	32	150	0.2799	0.9130 ± 0.0204
16	0.001	16	100	0.3917	0.9130 ± 0.0268
17	0.001	16	200	0.3572	0.9130 ± 0.0254
18	1.0	32	200	0.5054	0.9087 ± 0.0217
19	0.1	16	200	0.4145	0.9087 ± 0.0143
20	0.1	32	200	0.3833	0.9087 ± 0.0167

References

[1] Mark Hopkins, Erik Reeber, George Forman, and Jaap Suermondt. Spambase. UCI Machine Learning Repository, 1999. DOI: https://doi.org/10.24432/C53G6X.